A Multi-Objective Genetic Programming Approach with Self-Adaptive α Dominance to Uncertain Capacitated Arc Routing Problem

Shaolin Wang, Yi Mei, Mengjie Zhang
Victoria University of Wellington,
School of Engineering and Computer Science,
PO Box 600, Wellington, New Zealand
Email: {shaolin.wang, yi.mei, mengjie.zhang}@ecs.vuw.ac.nz

***Abstract*—The Uncertain Capacitated Arc Routing Problem (UCARP) has a variety of real-world applications. Genetic Programming Hyper-heuristic (GPHH) is considered a promising technique to handle UCARP. Many scholars have shown the power of GPHH of evolving effective routing policies. However, the size of the evolved routing policies is ignored. Typically, smaller routing policies can have better interpretability and generalisation. Thus, it is necessary to optimise the size along with the effectiveness. The objective selection bias issue arises as the size is much easier to be optimised than effectiveness. The Pareto front is biased to the size gradually during the evolutionary process. To address this issue, we develop an α dominance criteria based Multi-Objective GP with a self-adaptive α scheme (αMOGP-sa). The basic idea of the α dominance criteria is to set tradeoff rates between objectives. For different instances, the search space can be very different. In this case, the self-adaptive α scheme is employed to automatically tuning the α value during the evolutionary process so that we can identify a valid α value for different instances. This paper examines the proposed algorithm in eight different problem instances. The experimental results showed that αMOGP-sa could effectively handle the objective selection bias issue, and evolve much better Pareto front on Hyper-Volume and Inverted Generational Distance than the current state-of-the-art MOGP approach for UCARP in terms of effectiveness and size on all instances. Also, αMOGP-sa can evolve much smaller routing policies than the state-of-art single-objective GPHH without sacrificing effectiveness.**

I. Introduction

The Capacitated Arc Routing Problem (CARP), first proposed by Golden and Wong [1], is a famous combinatorial optimisation problem. CARP's primary goal is to serve a set of required edges (also called tasks) in a graph using a fleet of vehicles with minimum cost. CARP has been proven to be NP-hard [2] and has a variety of real-world applications, such as waste collection [3] and winter gritting [4]. CARP has received much research interest, and there have been extensive studies for handling it [2], [5]. However, most existing studies assumed that all the problem parameters (e.g., task demand) are fixed and known in advance, which are static problems. This is usually not true in reality. There are plenty of uncertainties in the real world. For example, the exact amount of waste to be collected in the waste collection problem is unknown in advance and can only be revealed during the waste collection process.

Uncertain Capacitated Arc Routing Problem (UCARP) [6] was proposed to reflect the reality better than CARP which are in nature dynamic problems. *Route failures* is one of the main challenges in UCARP. A route failure occurs when the remaining capacity of a vehicle is insufficient to serve the edge. In this case, the vehicle has to go back to the depot to refill and return to complete the service. This leads to a considerable recourse cost. Due to the challenge of handling route failures, traditional optimisation approaches designed for CARP, such as mathematical programming [1] and evolutionary algorithms [7], are not directly applicable to UCARP. These traditional optimisation approaches prefer optimising a robust solution beforehand and modify the preplanned solution by a recourse operator once route failures occur. However, it is difficult to optimise a single robust solution that can satisfy all the environments, and it is time-consuming to re-optimise the entire remaining solution in real-time.

The routing policy-based approach [8] is considered one of the most flexible methods for handling the uncertain environment in UCARP. In contrast to the traditional solution optimisation approaches, it does not require to pre-plan any solutions. The routing policy-based approach can respond to the uncertain environment immediately. Usually, a routing policy is a priority function, and vehicles use it to determine which task to serve based on the priority when the vehicle becomes idle [8]. There exist some manually designed routing policies (e.g. Path Scanning [7]) based on some existing constructive heuristics. However, the manually designed routing policies' effectiveness can be affected by a variety of factors, such as the graph topology, the objective(s) and the scenario [9]. To address this drawback, Genetic Programming Hyper-heuristic (GPHH) can be utilised to automatically evolve effective routing policies for UCARP [10], [11].

GPHH has shown its strength in automatically evolving effective routing policies for UCARP [12], [13], [14]. However, the size (number of nodes) of the GP-evolved routing policies is still ignored. Typically, smaller routing policies can have better generalisation and interpretability, but the

routing policies evolved by GPHH are typically very large. One of the main reasons is that GP tends to continuously increase the size of its individuals, known as *bloat* [15], [16]. Thus, it is necessary to optimise the size along with the effectiveness. However, it is not easy to simultaneously optimise effectiveness and size. One main challenge when simultaneously optimising the routing policies' effectiveness and size using a traditional Multi-Objective GP (MOGP) is that the size is much easier to be optimised than the effectiveness. As the evolutionary process continues, the population will be occupied by many small (but poor) routing policies gradually, and lose the exploration's ability. This is the objective selection bias issue. To handle the issue, Bleuler [17] propose a Strength Pareto Evolutionary Algorithm 2 (SPEA2) which use an archive to store Pareto front. Wang et al. [18] proposed a two-stage MOGP to set an anchor in the objective space to avoid the bias. Wang et al. [19] proposed a MOGP with different α schemes to balance two objectives. However, they can only partially handle the problem.

In this paper, the aim is to evolve compact and effective routing policies by simultaneously optimising the effectiveness and size. To address the objective selection bias issue, we will propose an α dominance criteria based Multi-Objective GP. Specifically, we propose a self-adaptive α scheme to automatically identify the bias and adjust the α value during the evolutionary process. We expect that the newly proposed α dominance criteria based MOGP with self-adaptive α scheme (αMOGP-sa) can find the suitable α value for each scenario and effectively handle the objective selection bias issue. Specifically, this paper has the following research objectives:

1) Develop a novel α dominance criterion based Multi-Objective GP with a self-adaptive α scheme (αMOGP-sa) to handle the issue of objective selection bias in UCARP;
2) Verify the performance of the αMOGP-sa on a set of UCARP instances and;
3) Verify the effectiveness of the proposed self-adaptive α scheme.

II. Background

A. Uncertain Capacitated Arc Routing Problem

Typically, a UCARP instance is based on a connected graph $G(V, E)$, where V is the set of vertices and E is the set of edges. All the edges are indirect, meaning that each edge can be interpreted as a pair of arcs with opposite directions. Generally, each edge $e \in E$ has a positive random deadheading cost $\tilde{dc}(e)$ which corresponding to the cost of traversing the edge. A subset of edges $T \subseteq E$ with positive random demand are called *task*, and each task $t \in T$ has a positive random demand $\tilde{d}(t)$ and a positive serving cost $sc(t)$. $v_0 \in V$ is the depot in the graph. A set of vehicles with capacity Q are allocated at the v_0 to serve the tasks in T. The problem's main goal is to minimise the total cost of serving all the tasks subject to the following three constraints. First, a vehicle must start and finish at the v_0. Second, the total demand served by a vehicle in a single trip cannot exceed the vehicle's capacity. Third, each task must be served exactly once, although it can be traversed multiple times.

Generally, a UCARP instance contains many random variables, i.e. the task demand $\tilde{d}(t)$ and deadheading cost $\tilde{dc}(e)$, each of which can have different samples. Accordingly, a UCARP instance may contain many different samples. UCARP has the following assumptions for the random variables. The distribution of the task demand and deadheading cost of an edge is known in advance. The actual deadheading cost is revealed after traversing over the edge, and the actual value of the task demand is realised when the task has been served. Each edge has a low probability of being absent from the graph, and this absence is not known until the vehicle reaches the head node of the edge.

Two uncontrollable failures may be encountered during the serving process because of the random variables in an uncertain environment. A route failure occurs when a vehicle's capacity is insufficient to serve a task because the actual demand exceeds the expected demand of the task. An edge failure occurs when the edge in the route becomes infeasible. Different strategies are employed to handle the two failures. A recourse operator is applied to repair route failures. When a route failure occurs, the recourse operator will lead the vehicle returns to the depot to refill and return to the failed task to complete the remaining service. When the next edge becomes inaccessible, a detour (e.g., use Dijkstra's algorithm to find the shortest path under the current situation) can be used to repair edge failures.

B. Related Work

CARP is a famous combinatorial optimisation problem and has been proven NP-hard [2]. A variety of studies have been made on dealing with CARP. *Exact* approaches, such as integer linear programming model [1] and cutting plane algorithm [20], can guarantee that the obtained solutions are optimal. However, these exact approaches are limited to only a small number of instances due to CARP's NP-hardness. They become time-consuming as the problem size becomes large. Recently, meta-heuristic approaches, such as Tabu search [21], [22], Genetic Algorithm [23], Memetic Algorithms [7], [24], [25], ant colony schema [26], and ant colony optimization [27], are utilised as they are much cheaper than exact approaches. However, they cannot apply to UCARP directly since they cannot handle the route failure effectively.

So far, there are two main types of approaches for solving UCARP, the proactive approaches [28], [29], [30] and the reactive approaches [10], [14], [12]. The main idea of the proactive approaches is to find robust solutions that are able to handle all the possible realisations of the random variables. Usually, there are two main components in the output of a proactive approach, i.e. a robust pre-planned solution and a recourse operator. A proactive approach can produce a robust solution based on the prediction of the environment. The robust solution is executed, and the recourse operator is utilised to handle the possible failures. The performance of a

proactive approach is highly associated with the accuracy of predicting the stochastic environment and the effectiveness of the recourse operators. The output of a reactive approach is an effective routing policy. The routing policy is applied as a heuristic to help decide which task to serve next when a vehicle is idle.

The reactive approaches' main idea is to use GPHH to evolve effective routing policies that construct the solution gradually in real-time. Weise et al. [31] first proposed a GPHH for evolving heuristics for stochastic CARP, and the results showed that the GP-evolved heuristics could outperform the manually designed heuristics. However, stochastic CARP only consider the random presence of tasks but ignore the stochastic demand. Liu et al. [10] utilised a GPHH for evolving routing policies for UCARP, and a filter mechanism was proposed to filter irrelevant candidate tasks during the decision-making process. Mei et al. [13] extended the model from the single-vehicle version to the multiple-vehicle version to reflect reality better. To make an improvement, MacLachlan et al. [12] proposed a novel no-refill task filtering method and a useful look-ahead terminal. Liu et al. [14] proposed a new predictive-reactive approach using cooperative coevolution GP to evolve task sequences and a recourse policy simultaneously. To improve efficiency when applying GPHH to new scenarios, Ardeh et al. [32] utilised transfer learning to speed up the training process.

In addition to effectiveness, size is also an important aspect of the GP-evolved routing policies. Usually, smaller routing policies can have better generalisation and interpretability. However, the GP-evolved routing policies are typically very large. One main reason is that GP tends to evolve increasingly larger individuals during the evolutionary process called *bloat*. Thus, it is necessary to control the tree size while ensuring effectiveness. The most straightforward way is to limit the GP tree size/depth [33]. However, it is challenging to set the limit properly without extensive trial and error[15]. The penalty term is another good way to control the GP tree size. It can be applied to penalise the larger trees which is usually called "parsimony pressure". Same as the size limitation, it is also different to set a proper weight for the penalty term without extensive trial and error [16], [34]. Evolutionary Multi-Objective Optimisation (EMO) approaches is considered a promising way to control the GP tree size [15], [35], [36]. It can find a Pareto front that each solution can satisfy different objectives, and users can choose solutions in the front based on their preference. There are many simple yet effective EMO approaches, such as NSGA-II [37] and SPEA2 [15].

One issue occurs when optimising the GP tree size and effectiveness simultaneously. The size is much easier to be optimised than the effectiveness. It is much easier to evolve small individual than the effective individual. Then, the Pareto front will be biased to the size [16], [38], [18], [19]. Bleuler [17] utilised the Strength Pareto Evolutionary Algorithm 2 (SPEA2) to handle this issue. However, SPEA2 does not work well in UCARP since it is hard to maintain the diversity of the population in a dynamic and stochastic problem [18]. This issue can also be partially addressed by Preference-Based EMO approaches [39], [40]. To our best knowledge, Preference-Based EMO approaches focus on some specific regions of the Pareto front. It can obtain a Pareto front in the "preferred region" rather than the Pareto front in the whole search space, but our goal of this paper is to find the entire Pareto front in the search space because we want to provide the users as many choices as possible so that the users can choose the solution that works best for them. Wang et al. [18] proposed a two-stage MOGP to deal with this issue. The solution from the first stage is used as an anchor to avoid bias. However, the Pareto front still biases to the size gradually. It only slows down the bias process. Wang et al. [19] proposed a MOGP using α dominance criteria with three different α schemes. The results showed that α dominance criteria could effectively handle the bias issue. However, it is hard to find one manually designed scheme that can satisfy all scenarios. Thus, it is necessary to adjust the alpha value during the evolutionary process automatically.

III. The New Approach

A. Overall Framework

Fig. 1 shows the diagram of the αMOGP-sa. The basic idea of αMOGP-sa is using α-dominance criteria to replace the traditional dominance criteria. After evaluating each routing policy, the population will be non-dominated sorted based on α dominance. Rank and sparsity will be assigned to each individual in the population. Then, we can detect the bias of the Pareto front of the current generation and adjust the α value. After that, a new population will be generated. The tournament selection will be applied to select parent individuals based on each candidate individual's rank and sparsity. The $crossover$ and $mutation$ operators will be applied to breed a new population. When the stopping criteria are reached, the final Pareto front based on traditional dominance criteria will be returned.

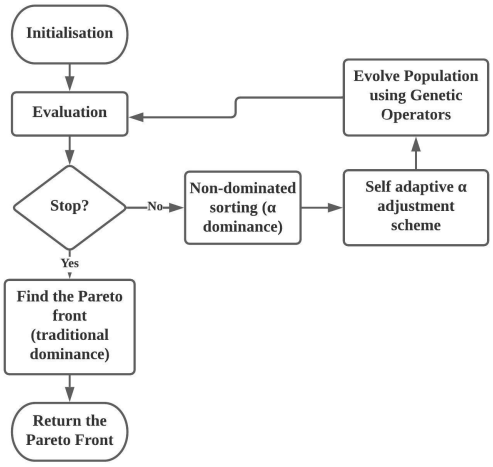


Fig. 1. The diagram of αMOGP-sa.

B. Individual Representation

In αMOGP-sa, all the routing policies in the population are essentially represented as arithmetic priority functions. Each priority function is a combination of the state features. For example, a priority function is "SC + DEM", SC refers to the expected serving cost of the candidate task and DEM refers to the expected demand of the candidate task. Fig. 2 shows a simple routing policy. Each routing policy will be applied as a decision-maker to construct a solution in a simulation process. The routing policy is applied to each unserved task and calculate the priority value. The task with the best priority value will be served next once a vehicle becomes idle. After all the tasks have been served, the set of routes of the vehicles are returned as the solution constructed by the routing policy.

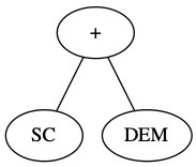


Fig. 2. A representation of GP tree.

C. Fitness Evaluation

In the evaluation process, the effectiveness and the size should be quantified. Given a routing policy rp, the $size(rp)$ can be simply quantified by counting the number of nodes in a routing policy. To quantify the effectiveness, the rp should be passed to a series of simulation processes. $eff(rp)$ is defined as the average quality (i.e. total cost) of the solution constructed by rp over the set of simulation processed $\mathcal{T}$. Specifically,

$$eff(rp) = \frac{1}{|\mathcal{T}|} \sum_{t \in \mathcal{T}} tc(rp, t), \tag{1}$$

where $tc(rp, t)$ refers to the total cost of the set of routes obtained by rp on sample t and $|\mathcal{T}|$ stands for the number of instance samples.

In this paper, we randomly re-sample the subset of training samples in the fitness evaluation process. Such training sample rotation operation has been commonly used in literature and demonstrated to be able to improve the generalisation of the evolved routing policies [10], [41].

D. Self-Adaptive α Scheme

The basic idea of the α-dominance criteria is to set tradeoff rates between objectives. For our problem to minimise effectiveness (o_1) and size (o_2) simultaneously, a feasible solution x dominates the other solution y using α-dominance criteria can be formulated as the follows:

$$\min(o_1,\ o_2), \tag{2}$$

$$\delta_i(x, y) \leq 0,\ \forall i \in \{1, 2\},\ \wedge \tag{3}$$

$$\delta_i(x, y) < 0,\ \exists i \in \{1, 2\} \tag{4}$$

where

$$\delta_i(x, y) = f_i(x) - f_i(y) + \sum_{j \neq i}^{1 \ldots 2} \alpha_{ij}(f_j(x) - f_j(y)) \tag{5}$$

When all α_{ij} values equals to 0, α dominance becomes traditional dominance. More details of the α dominance can be investigated in [42]. One of the main challenges of the α dominance is to find a proper α value [19]. For different instances, we need different α value as the search space can be quite different, and the range of the objectives can also be quite different for different instances. In this paper, we propose a self-adaptive α scheme. The scheme will first identify the upper and lower boundary of the effectiveness and size to know the range of the objective space, and if the new boundary is greater than the original, then the original boundary is replaced by the new boundary. We assume that a good Pareto front should cover the boundary as much as possible. Then, the current Pareto front based on traditional dominance criteria is utilised to identify the bias. If the current front is biased to effectiveness, the α should be decreased. If the current front is biased to size, the α should be increased. The pseudocode of the self-adaptive α scheme is shown in Algorithm 1.

IV. Experimental Studies

To evaluate the performance of the αMOGP-sa, we test all compared algorithms on a set of UCARP instances. We compare the αMOGP-sa with SPEA2, TSMOGP [18] and the α-MOGP [19] which is the current state-of-the-art MOGP approach for UCARP in terms of effectiveness and size.

A. Experiment Setup

We select eight commonly used UCARP instances to evaluate the performance of the proposed approach. The problem size varies from small (22 tasks and 5 vehicles) to large (97 tasks and 10 vehicles). The experiment consists of two phases, i.e. training and test. In the training phase, routing policies are trained based on the training set. There are 5 training samples during the evaluation, and they are re-sampled each generation. In the test phase, the routing policy will be tested on 500 unseen test samples, which can avoid testing bias.

The terminal set is shown in Table I. The function set is $\{+, -, \times, /, \min, \max\}$. The "/" operator is protected divide which returns 1 if divided by 0, are applied in the experiments. Both terminal set and function set are commonly used in GPHH for UCARP literature [10], [12], [14], [19]. For all compared algorithms, the generation is 50, and the population size is 1000. Ramp-half-and-half initialisation is used. The ratio of crossover to mutation is 0.85 : 0.15. The tournament selection size is 7 for all compared algorithms. The maximal depth is 8, which has been commonly used in previous studies (e.g., [10]). Elitism size for all compared approaches is 10. There is no elitism parameter for SPEA2 as it has an archive to store elites. When α value equals to 0, the α dominance criteria is the traditional dominance criteria. Previous studies [18], [19] have demonstrated the existence of objective selection bias issue when using traditional dominance criteria. To test

Algorithm 1: The self-adaptive α scheme.

Input: The population pop, The α value, Upper boundary and lower boundary of effectiveness u_{eff} and l_{eff}, Upper boundary and lower boundary of size u_{size} and l_{size}, learning rate lr

Output: The α value, Upper and lower boundary of effectiveness u_{eff} and l_{eff}, Upper and lower boundary of size u_{size} and l_{size}

find the current upper and lower boundary of effectiveness and size u'_{eff}, l'_{eff}, u'_{size}, l'_{size} from pop;
if $u'_{eff} > u_{eff}$ **then**
  $u_{eff} = u'_{eff}$;
end
if $l'_{eff} < l_{eff}$ **then**
  $l_{eff} = l'_{eff}$;
end
if $u'_{size} > u_{size}$ **then**
  $u_{size} = u'_{size}$;
end
if $l'_{size} < l_{size}$ **then**
  $l_{size} = l'_{size}$;
end
find the current Pareto front pf using traditional dominance criteria from pop;
Calculate the average mean effectiveness avg_{eff} of the pf;
Calculate the average mean size avg_{size} of the pf;
if $\frac{u_{eff}-avg_{eff}}{avg_{eff}-l_{eff}} > 1.0$ *and* $\frac{u_{size}-avg_{size}}{avg_{size}-l_{size}} < 1.0$ **then**
  `// bias to effectiveness, then decrease the` α
  $\alpha = \alpha - lr$;
end
if $\frac{u_{eff}-avg_{eff}}{avg_{eff}-l_{eff}} < 1.0$ *and* $\frac{u_{size}-avg_{size}}{avg_{size}-l_{size}} > 1.0$ **then**
  `// bias to size, then increase the` α
  $\alpha = \alpha + lr$;
end
return α,u_{eff}, l_{eff},u_{size},l_{size};

our hypothesis that the self-adaptive α scheme can detect the bias and find a suitable α value to address the bias, the α value is set to 0 at beginning for αMOGP-sa. The learning rate for the self-adaptive α scheme is 0.2. The Evolutionary Computation Java (ECJ) package [43] is utilised to implement all the algorithms. The results are collected based on 30 independent runs for each algorithm on each UCARP instance.

B. Results and Discussions

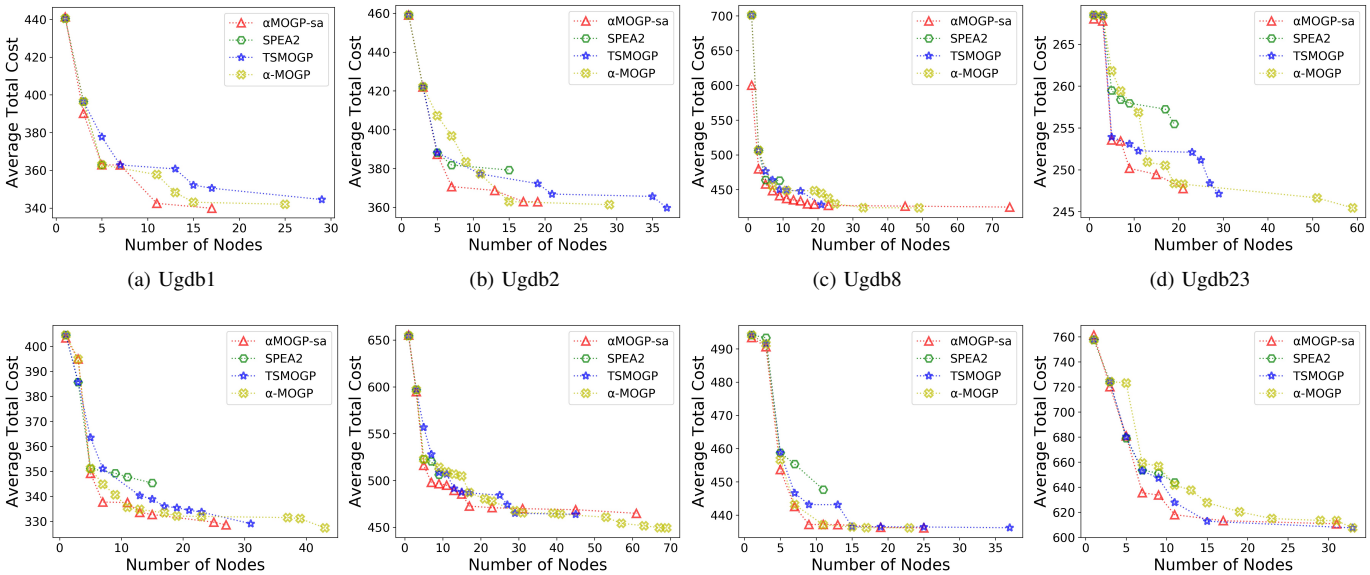
The Wilcoxon rank sum test with a significance level of 0.05 is used to compare all algorithms. For each compared algorithm, if the proposed algorithm is statistically significantly better than, worse than, and comparable with that of the compared algorithm, the corresponding entry is marked with (+), (-), or (=), respectively.

TABLE I
THE TERMINAL SET IN THE EXPERIMENTS.

Terminal	Description
CTD	cost from the depot to the candidate task
CR	cost from the current location to the depot
CTT1	cost from the candidate to its closest remaining task
CFR1	cost from the closest other route to the candidate task
CFD	cost from the head node of the task to the depot
CFH	cost from the candidate task to the current location
DEM	expected demand of the candidate task
DEM1	demand of unserved task that closest to the candidate task
FULL	fullness (served demand over capacity) of the vehicle
FUT	fraction of unassigned tasks
FRT	fraction of unserved tasks
RQ	remaining capacity of the vehicle
RQ1	remaining capacity of the closest other route to the candidate task
SC	cost of serving the candidate task
ERC	a random constant value

We adopt two commonly used indicators for multi-objective optimisation, i.e. Hyper-Volume (HV) and Inverted Generational Distance (IGD). The results are shown in Tables II and III. It can be seen that the αMOGP-sa perform best among all compared algorithms on almost all instances. This is as expected as the self-adaptive α scheme can automatically find a suitable α value for different scenarios. With the suitable α value, αMOGP-sa can generate the better Pareto front. To make the results more obvious, the final Pareto front over the 30 independent runs on each instance is shown in Fig. 3. We can see that the Pareto fronts of the αMOGP-sa are better distributed in the objective space. The Pareto fronts of α-MOGP is close to that of αMOGP-sa. This is expected as both algorithms utilised α dominance instead of traditional dominance. The Pareto fronts of αMOGP-sa are better than that of α-MOGP. This can also indicate the effectiveness of the self-adaptive α scheme.

In this paper, we are still concerned about effectiveness because we would like some effective and compact routing policies. In this case, we extract the individual with the best average total cost in the final Pareto front from each run of each MOGP algorithm. We also compare our algorithm with the state-of-art single-objective GPHH approach. Table IV shows the mean and standard deviation of the average total cost (effectiveness) of the compared algorithms. It can be seen that αMOGP-sa can perform comparable with GPHH on all instances. Besides, it can outperform SPEA2 on all instances because SPEA2 has the objective selection bias issue on size. This is consistent with the pattern of the final Pareto front of the SPEA2 in Fig. 3. αMOGP-sa can outperform TSMOGP on 3 out 8 instances and achieve comparable effectiveness on other instances. αMOGP-sa performs slightly worse than α-MOGP on 1 instance, but it performs significantly better than α-MOGP on 3 out of 8 instances and comparable on other instances. Table V shows the mean and standard deviation of the size of compared algorithms. αMOGP-sa can evolve significantly smaller routing policies than GPHH. This is because



(a) Ugdb1 (b) Ugdb2 (c) Ugdb8 (d) Ugdb23

(e) Uval9A (f) Uval9D (g) Uval10A (h) Uval10D

Fig. 3. The final Pareto front over 30 independent run of compared algorithms.

TABLE II
THE MEAN AND STANDARD DEVIATION FOR **HV** OF THE COMPARED ALGORITHMS IN TEST PROCESS.

Instance	**SPEA2**	**TSMOGP**	**α-MOGP**	**αMOGP-sa**
Ugdb1	0.8625(0.04)	0.937(0.03)	0.9385(0.03)	0.9578(0.02)(+)(+)(+)
Ugdb2	0.8902(0.04)	0.9334(0.02)	0.9316(0.03)	0.954(0.01)(+)(+)(+)
Ugdb8	0.8392(0.05)	0.9274(0.04)	0.9284(0.03)	0.9705(0.01)(+)(+)(+)
Ugdb23	0.866(0.03)	0.9274(0.02)	0.9286(0.04)	0.9478(0.01)(+)(+)(+)
Uval9A	0.9577(0.02)	0.9815(0.0)	0.9735(0.02)	0.9887(0.0)(+)(+)(+)
Uval9D	0.8524(0.05)	0.9477(0.01)	0.9326(0.04)	0.9592(0.01)(+)(+)(+)
Uval10A	0.9542(0.02)	0.9863(0.01)	0.9747(0.02)	0.9931(0.0)(+)(+)(+)
Uval10D	0.9019(0.03)	0.9684(0.01)	0.9566(0.04)	0.9775(0.01)(+)(+)(+)

TABLE III
THE MEAN AND STANDARD DEVIATION FOR **IGD** OF THE COMPARED ALGORITHMS IN TEST PROCESS.

Instance	**SPEA2**	**TSMOGP**	**α-MOGP**	**αMOGP-sa**
Ugdb1	0.0921(0.03)	0.0678(0.03)	0.066(0.04)	0.0345(0.01)(+)(+)(+)
Ugdb2	0.1329(0.03)	0.0892(0.03)	0.0865(0.03)	0.048(0.01)(+)(+)(+)
Ugdb8	0.1359(0.04)	0.0973(0.03)	0.1113(0.05)	0.0244(0.01)(+)(+)(+)
Ugdb23	0.1207(0.02)	0.0788(0.02)	0.0851(0.05)	0.047(0.01)(+)(+)(+)
Uval9A	0.1218(0.03)	0.0522(0.01)	0.0598(0.02)	0.0486(0.02)(+)(=)(+)
Uval9D	0.202(0.05)	0.0842(0.02)	0.0807(0.04)	0.0545(0.02)(+)(+)(+)
Uval10A	0.097(0.03)	0.0444(0.02)	0.0466(0.02)	0.0271(0.01)(+)(+)(+)
Uval10D	0.1033(0.03)	0.0522(0.03)	0.0674(0.05)	0.0316(0.01)(+)(+)(+)

αMOGP-sa considers both size and effectiveness during the evolutionary process. We can see that SPEA2 obtains much smaller routing policies than αMOGP-sa. This is because of the objective selection bias issue. All the routing policies obtained by SPEA2 are small but poor. This is consistent with the results in Table IV. The effectiveness of SPEA2 is significantly worse than αMOGP-sa. αMOGP-sa can achieve comparable size with TSMOGP and α-MOGP on 6 out of 8 instances. Besides, αMOGP-sa outperforms TSMOGP on Ugdb2, and it also outperforms α-MOGP on Ugdb23.

C. Further Analysis

To further verify the self-adaptive α scheme's effectiveness, we also compare the NSGP-II with the self-adaptive α scheme (NSGP-II-sa) with the traditional NSGP-II and a Two-Stage NSGP-II (TSNSGP-II). From Table VI, we can see that the NSGP-II-sa can outperform the other two algorithms without self-adaptive α scheme. This is expected as the scheme can automatically adjust the dominance criteria to reduce the objective selection bias to generate better Pareto front. The IGD results are consistent with HV. Because of the page limit. The table on IGD is omitted here.

We also plot the convergence curve of α value on all

TABLE IV
THE MEAN AND STANDARD DEVIATION OF THE **AVERAGE TOTAL COST** OF THE BEST INDIVIDUAL FROM COMPARED ALGORITHMS.

Instance	GPHH	SPEA2	TSMOGP	α-MOGP	αMOGP-sa
Ugdb1	355.4678(14.83)	389.147(15.56)	356.4119(10.74)	354.7738(12.05)	351.8234(7.41)(=)(+)(=)(=)
Ugdb2	371.7201(7.67)	404.084(15.62)	372.0391(9.14)	370.1717(7.91)	372.9289(6.03)(=)(+)(=)(-)
Ugdb8	430.3441(8.16)	509.8187(29.91)	451.9689(22.96)	441.0539(10.75)	433.7355(5.58)(=)(+)(+)(+)
Ugdb23	252.4679(3.11)	262.3478(3.66)	253.0202(3.27)	250.4665(2.16)	251.2397(1.95)(=)(+)(+)(=)
Uval9A	335.1342(3.8)	371.2449(19.77)	336.7092(4.57)	336.0367(3.65)	333.8784(2.22)(=)(+)(+)(+)
Uval9D	478.1405(16.68)	586.5754(51.35)	480.6811(10.14)	474.2416(11.94)	477.6783(7.09)(=)(+)(=)(=)
Uval10A	439.4139(5.97)	481.591(16.89)	442.041(10.9)	440.5193(3.92)	438.1817(2.47)(=)(+)(=)(+)
Uval10D	620.9134(7.97)	699.7957(32.83)	624.1609(8.61)	619.5877(8.45)	620.2146(4.96)(=)(+)(=)(=)

TABLE V
THE MEAN AND STANDARD DEVIATION OF THE **AVERAGE NUMBER OF NODES** OF THE BEST INDIVIDUAL FROM COMPARED ALGORITHMS.

Instance	GPHH	SPEA2	TSMOGP	α-MOGP	αMOGP-sa
Ugdb1	74.6(23.84)	8.4(5.18)	30.6(18.01)	27.67(15.97)	34.47(31.68)(+)(-)(=)(=)
Ugdb2	71.93(23.79)	5.73(3.22)	38.33(19.8)	28.8(13.88)	28.07(13.26)(+)(-)(+)(=)
Ugdb8	65.47(24.33)	5.6(4.49)	42.27(36.62)	51.67(19.03)	51.2(33.63)(+)(-)(=)(=)
Ugdb23	71.8(25.22)	8.6(4.53)	31.33(19.58)	47.53(26.57)	37.07(36.73)(+)(-)(=)(+)
Uval9A	56.93(18.27)	8.53(4.83)	30.6(13.72)	28.4(14.79)	32.27(19.78)(+)(-)(=)(=)
Uval9D	69.27(29.46)	7.53(6.15)	42.67(19.31)	58.0(29.24)	40.7(16.45)(+)(-)(=)(+)
Uval10A	60.47(18.59)	4.53(4.54)	24.07(10.86)	19.27(9.74)	25.8(13.99)(+)(-)(=)(-)
Uval10D	65.33(14.35)	9.2(5.18)	35.0(21.22)	47.67(23.7)	42.4(18.06)(+)(-)(-)(=)

TABLE VI
THE EXPERIMENT RESULT (HV) TO SHOW THE EFFECTIVENESS OF THE SELF-ADAPTIVE α SCHEME.

Instance	NSGP-II	TS-NSGP-II	NSGP-II-sa
Ugdb1	0.7512(0.06)	0.7929(0.06)	0.9082(0.04)(+)(+)
Ugdb2	0.8342(0.03)	0.8629(0.04)	0.9261(0.02)(+)(+)
Ugdb8	0.8707(0.03)	0.8798(0.04)	0.9508(0.03)(+)(+)
Ugdb23	0.5643(0.08)	0.6(0.09)	0.8213(0.04)(+)(+)
Uval9A	0.728(0.06)	0.8074(0.06)	0.8939(0.02)(+)(+)
Uval9D	0.7125(0.07)	0.794(0.06)	0.8851(0.02)(+)(+)
Uval10A	0.5867(0.1)	0.7001(0.09)	0.9001(0.03)(+)(+)
Uval10D	0.6329(0.14)	0.7697(0.11)	0.9228(0.02)(+)(+)

instances on 30 independent runs. The convergence curve is shown in Fig. 4. We can see that the curves grow rapidly at first, and as generation increases, the curves stabilise and eventually converge. Besides, the curve eventually converges at different values for different instances. For Ugdb23 and Uval10A, the α value converge to large values. For Uval9A, α value converges to a medium value. For the rest of the instances, α value converge to small values. This is consistent with the assumption that α value is scenario specified. The self-adaptive α scheme is capable of identifying suitable α values for different scenarios. This is also why the αMOGP-sa can outperform α-MOGP on HV and IGD.

V. CONCLUSIONS

This paper aims to evolve compact and effective routing policies for UCARP using Multi-Objective Optimisation. The objective selection bias issue occurs when optimising the size along with the effectiveness. To handle the issue, we proposed a simple yet effective α dominance criterion based Multi-Objective GP with a self-adaptive α scheme (αMOGP-sa). The experimental results showed that the newly proposed (αMOGP-sa) could evolve much smaller routing policies without losing effectiveness than the state-of-art GPHH approach.

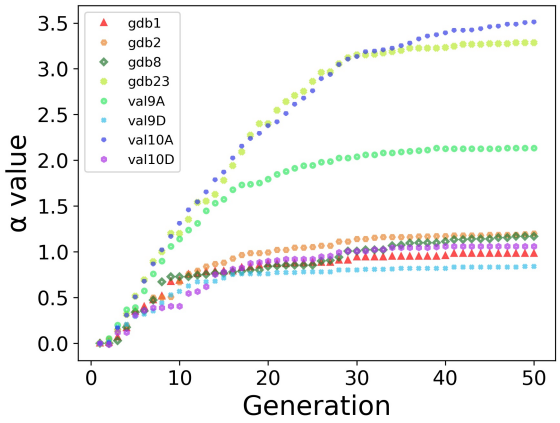


Fig. 4. The convergence curve of α value.

The αMOGP-sa can also generate much better Pareto fronts in terms ofHV and IGD than other MOGP approaches from the literature. We also employ the newly proposed self-adaptive α scheme on NSGP-II. The experiment results indicate that the scheme can effectively address the objective selection bias issue. The convergence curve of α value indicates that the self-adaptive α scheme can find proper α values for different instances. In the future, we will try alternative promising ways to adjust the α value during the evolutionary process, and develop new approaches that can further reduce the size of the evolved policies.

REFERENCES

[1] B. L. Golden and R. T. Wong, "Capacitated arc routing problems," *Networks*, vol. 11, no. 3, pp. 305–315, 1981.

[2] S. Wøhlk, "A decade of capacitated arc routing," in *The vehicle routing problem: latest advances and new challenges*. Springer, 2008, pp. 29–48.

[3] S. Amponsah and S. Salhi, "The investigation of a class of capacitated arc routing problems: The collection of garbage in developing countries," *Waste Management*, vol. 24, no. 7, pp. 711–721, 2004.

[4] H. Handa, L. Chapman, and X. Yao, "Robust route optimization for gritting/salting trucks: a cercia experience," *IEEE Computational Intelligence Magazine*, vol. 1, no. 1, pp. 6–9, 2006.

[5] M. Dror, *Arc routing: theory, solutions and applications*. Springer Science & Business Media, 2012.

[6] Y. Mei, K. Tang, and X. Yao, "Capacitated arc routing problem in uncertain environments," in *IEEE Congress on Evolutionary Computation*. IEEE, 2010, pp. 1–8.

[7] P. Lacomme, C. Prins, and W. Ramdane-Cherif, "Competitive memetic algorithms for arc routing problems," *Annals of Operations Research*, vol. 131, no. 1-4, pp. 159–185, 2004.

[8] U. Ritzinger, J. Puchinger, and R. F. Hartl, "A survey on dynamic and stochastic vehicle routing problems," *International Journal of Production Research*, vol. 54, no. 1, pp. 215–231, 2016.

[9] J. Jacobsen-Grocott, Y. Mei, G. Chen, and M. Zhang, "Evolving heuristics for dynamic vehicle routing with time windows using genetic programming," in *IEEE Congress on Evolutionary Computation*. IEEE, 2017, pp. 1948–1955.

[10] Y. Liu, Y. Mei, M. Zhang, and Z. Zhang, "Automated heuristic design using genetic programming hyper-heuristic for uncertain capacitated arc routing problem," in *Proceedings of the Genetic and Evolutionary Computation Conference*. ACM, 2017, pp. 290–297.

[11] H. Al-Sahaf, Y. Bi, Q. Chen, A. Lensen, Y. Mei, Y. Sun, B. Tran, B. Xue, and M. Zhang, "A survey on evolutionary machine learning," *Journal of the Royal Society of New Zealand*, vol. 49, no. 2, pp. 205–228, 2019.

[12] J. MacLachlan, Y. Mei, J. Branke, and M. Zhang, "An improved genetic programming hyper-heuristic for the uncertain capacitated arc routing problem," in *Australasian Joint Conference on Artificial Intelligence*. Springer, 2018, pp. 432–444.

[13] Y. Mei and M. Zhang, "Genetic programming hyper-heuristic for multi-vehicle uncertain capacitated arc routing problem," in *Proceedings of the Genetic and Evolutionary Computation Conference Companion*. New York, NY, USA: ACM, 2018, pp. 141–142.

[14] Y. Liu, Y. Mei, M. Zhang, and Z. Zhang, "A predictive-reactive approach with genetic programming and cooperative co-evolution for uncertain capacitated arc routing problem," *Evolutionary Computation*, 2019.

[15] S. Bleuler, M. Brack, L. Thiele, and E. Zitzler, "Multiobjective genetic programming: Reducing bloat using spea2," in *Proceedings of the Congress on Evolutionary Computation*. IEEE, 2001, pp. 536–543.

[16] E. D. De Jong, R. A. Watson, and J. B. Pollack, "Reducing bloat and promoting diversity using multi-objective methods," in *Proceedings of the 3rd Annual Conference on Genetic and Evolutionary Computation*. Morgan Kaufmann Publishers Inc., 2001, pp. 11–18.

[17] S. Bleuler, J. Bader, and E. Zitzler, "Reducing bloat in gp with multiple objectives," in *Multiobjective Problem Solving from Nature*. Springer, 2008, pp. 177–200.

[18] S. Wang, Y. Mei, J. Park, and M. Zhang, "A two-stage genetic programming hyper-heuristic for uncertain capacitated arc routing problem," in *2019 IEEE Symposium Series on Computational Intelligence (SSCI)*. IEEE, 2019, pp. 1606–1613.

[19] S. Wang, Y. Mei, and M. Zhang, "A multi-objective genetic programming hyper-heuristic approach to uncertain capacitated arc routing problems," in *2020 IEEE Congress on Evolutionary Computation (CEC)*. IEEE, 2020, pp. 1–8.

[20] J. M. Belenguer and E. Benavent, "A cutting plane algorithm for the capacitated arc routing problem," *Computers & Operations Research*, vol. 30, no. 5, pp. 705–728, 2003.

[21] R. W. Eglese and L. Y. Li, "A tabu search based heuristic for arc routing with a capacity constraint and time deadline," in *Meta-Heuristics*. Springer, 1996, pp. 633–649.

[22] A. Hertz, G. Laporte, and M. Mittaz, "A tabu search heuristic for the capacitated arc routing problem," *Operations research*, vol. 48, no. 1, pp. 129–135, 2000.

[23] P. Lacomme, C. Prins, and W. Ramdane-Chérif, "A genetic algorithm for the capacitated arc routing problem and its extensions," in *Workshops on Applications of Evolutionary Computation*. Springer, 2001, pp. 473–483.

[24] K. Tang, Y. Mei, and X. Yao, "Memetic algorithm with extended neighborhood search for capacitated arc routing problems," *IEEE Transactions on Evolutionary Computation*, vol. 13, no. 5, pp. 1151–1166, 2009.

[25] Y. Mei, K. Tang, and X. Yao, "Decomposition-based memetic algorithm for multiobjective capacitated arc routing problem," *IEEE Transactions on Evolutionary Computation*, vol. 15, no. 2, pp. 151–165, 2011.

[26] P. Lacomme, C. Prins, and A. Tanguy, "First competitive ant colony scheme for the carp," in *International Workshop on Ant Colony Optimization and Swarm Intelligence*. Springer, 2004, pp. 426–427.

[27] K. F. Doerner, R. F. Hartl, V. Maniezzo, and M. Reimann, "Applying ant colony optimization to the capacitated arc routing problem," in *International Workshop on Ant Colony Optimization and Swarm Intelligence*. Springer, 2004, pp. 420–421.

[28] G. Fleury, P. Lacomme, and C. Prins, "Evolutionary algorithms for stochastic arc routing problems," in *Workshops on Applications of Evolutionary Computation*. Springer, 2004, pp. 501–512.

[29] J. Wang, K. Tang, and X. Yao, "A memetic algorithm for uncertain capacitated arc routing problems," in *2013 IEEE Workshop on Memetic Computing (MC)*. IEEE, 2013, pp. 72–79.

[30] J. Wang, K. Tang, J. A. Lozano, and X. Yao, "Estimation of the distribution algorithm with a stochastic local search for uncertain capacitated arc routing problems," *IEEE Transactions on Evolutionary Computation*, vol. 20, no. 1, pp. 96–109, 2016.

[31] T. Weise, A. Devert, and K. Tang, "A developmental solution to (dynamic) capacitated arc routing problems using genetic programming," in *Proceedings of the 14th annual conference on Genetic and evolutionary computation*. ACM, 2012, pp. 831–838.

[32] M. A. Ardeh, Y. Mei, and M. Zhang, "Genetic programming hyper-heuristic with knowledge transfer for uncertain capacitated arc routing problem," in *Proceedings of the Genetic and Evolutionary Computation Conference Companion (GECCO)*. IEEE, jul 2019, pp. 334–335.

[33] R. Poli, W. B. Langdon, N. F. McPhee, and J. R. Koza, *A field guide to genetic programming*. Lulu. com, 2008.

[34] T. Soule and J. A. Foster, "Effects of code growth and parsimony pressure on populations in genetic programming," *Evolutionary computation*, vol. 6, no. 4, pp. 293–309, 1998.

[35] S. Chand, H. Singh, and T. Ray, "Evolving heuristics for the resource constrained project scheduling problem with dynamic resource disruptions," *Swarm and evolutionary computation*, vol. 44, pp. 897–912, 2019.

[36] Q. N. Huynh, S. Chand, H. K. Singh, and T. Ray, "Genetic programming with mixed-integer linear programming-based library search," *IEEE Transactions on Evolutionary Computation*, vol. 22, no. 5, pp. 733–747, 2018.

[37] K. Deb, S. Agrawal, A. Pratap, and T. Meyarivan, "A fast elitist non-dominated sorting genetic algorithm for multi-objective optimization: Nsga-ii," in *International conference on parallel problem solving from nature*. Springer, 2000, pp. 849–858.

[38] W. B. Langdon and J. Nordin, "Seeding genetic programming populations," in *European Conference on Genetic Programming*. Springer, 2000, pp. 304–315.

[39] L. Thiele, K. Miettinen, P. J. Korhonen, and J. Molina, "A preference-based evolutionary algorithm for multi-objective optimization," *Evolutionary computation*, vol. 17, no. 3, pp. 411–436, 2009.

[40] K. Deb and J. Sundar, "Reference point based multi-objective optimization using evolutionary algorithms," in *Proceedings of the 8th annual conference on Genetic and evolutionary computation*. ACM, 2006, pp. 635–642.

[41] T. Hildebrandt, J. Heger, and B. Scholz-Reiter, "Towards improved dispatching rules for complex shop floor scenarios: a genetic programming approach," in *Proceedings of Genetic and Evolutionary Computation Conference*. ACM, 2010, pp. 257–264.

[42] K. Ikeda, H. Kita, and S. Kobayashi, "Failure of pareto-based moeas: Does non-dominated really mean near to optimal?" in *Proceedings of the 2001 Congress on Evolutionary Computation (IEEE Cat. No. 01TH8546)*, vol. 2. IEEE, 2001, pp. 957–962.

[43] S. Luke, L. Panait, G. Balan, S. Paus, Z. Skolicki, J. Bassett, R. Hubley, and A. Chircop, "Ecj: A java-based evolutionary computation research system," *Downloadable versions and documentation can be found at the following url: http://cs. gmu. edu/eclab/projects/ecj*, 2006.